

Draaiboek snapshot-restore

Optie B — substantiële verbetering

Goyer Golf MP Ladder · v3.0.0-11.31 → toekomstige versie

Dit document beschrijft alle stappen die nodig zijn om de snapshot/restore-functionaliteit volledig werkend te maken. Het is geschreven als naslagwerk: iemand die deze app ooit eerder heeft gezien moet hiermee in één sessie de implementatie kunnen voltooien.

Huidige situatie (v11.31)

- UI 'Ladder herstellen' is verborgen via `display:none` op `index.html` regel ~1430
- Automatische snapshot-creatie bij partij-bevestiging werkt nog steeds
- `herstelSnapshot` heeft minimale fix (optie A): `synct standen/{uid}`
- Andere problemen (`actievePartijen`, `cfg`, `transactie`) blijven bestaan

De twee datalagen

Voor begrip van de wijzigingen is het cruciaal om de tweelagige data-architectuur te onthouden. Elke ladder heeft data op twee plekken in Firestore:

Laag	Locatie	Inhoud	Wie leest dit?
1 — Legacy	<code>ladders/{id}.spelers[]</code>	Volledige array met id, naam, rank, <code>hcp</code> , <code>partij</code> , <code>bevestiging</code> , <code>prevRank</code>	Adaptatie van beheer op <code>ladders/{id}.spelers[]</code>
2 — View	<code>ladders/{id}/standen/{uid}</code>	Per-speler doc met rank, partijen, gewonnen, <code>prevRank</code>	Adaptatie van beheer op <code>ladders/{id}/standen/{uid}</code>

Bij elke wijziging die ranks raakt, moeten **beide** lagen gesynchroniseerd worden. Voorbeelden waar dit al wel gebeurt: `bevestigUitslag` (`ronde.js`), `bevestigBeheerUitslag` (`uitslagen.js`), `slaStandOp` (`beheer.js`, sinds v11.28), `herstelSnapshot` (`beheer.js`, sinds v11.31, optie A).

Scope van optie B

Optie B verbreedt zowel **wat een snapshot bevat** als **wat herstel terugzet**. Doel: een snapshot vertegenwoordigt de volledige ladder-staat van een moment, en herstel zet die staat compleet terug.

Wat wel meegenomen wordt

- `spelers[]` — al bestaand, blijft hetzelfde formaat
- `cfg` — ladder-instellingen (`laagStijg`, `laagZak`, `hoogStijg`, `hoogZak`, `drempel`, `verliezerNaarWinnaar`)
- `actievePartijen[]` — lopende partijen op moment van snapshot
- `spelerIds[]` — uid-lijst zoals op de ladder staat
- `standen[]` — kopie van alle `standen/{uid}` docs (`rank`, `partijen`, `gewonnen`, `prevRank`)
- `nextId` — counter voor nieuwe spelers (wenselijk maar niet kritiek)

Wat NIET meegenomen wordt

- **uitslagen/ collectie** — historische scorekaarten zijn historie, geen state
- **spelers/{uid} collectie** — accounts blijven onveranderd
- **snapshots/ collectie** — de snapshots zelf, om recursie te voorkomen
- **Knockout-bracket data** — heeft eigen herstel-logica nodig (aparte sessie)
- **Toernooi-data** — losse module met eigen lifecycle

Stappenplan

Werk de volgende stappen in volgorde af. Elke stap heeft een eigen verificatie. Wijk niet af van de volgorde — een latere stap leunt op resultaten van eerdere.

Stap 1: Snapshot uitbreiden (slaSnapshotOp)

File: `js/beheer.js` · regel ~399

Deze functie wordt aangeroepen bij elke partij-bevestiging en toernooi-afronding. Breid de payload uit met de nieuwe velden. Zorg dat het ophalen van standen-subcollectie hier gebeurt, niet later.

Pseudocode voor de nieuwe payload:

```
async function slaSnapshotOp(label) {
  try {
    // Ophalen huidige ladder-data (we hebben deze nodig voor cfg, spelerIds,
    // etc.)
    const { data: ladderData } = await getLadderData(activeLadderId);

    // Ophalen alle standen/{uid} docs
    const standenSnap = await getDocs(
      collection(db, 'ladders', activeLadderId, 'standen')
    );
    const standen = {};
    standenSnap.forEach(d => standen[d.id] = d.data());

    // Opruiming oude snapshots (bestaande logica)
    const dertig = Date.now() - 30 * 24 * 60 * 60 * 1000;
    const oudeSnaps = await getDocs(
      query(SNAPSHOTS_COL, where('timestamp', '<', dertig))
    );
    oudeSnaps.forEach(d => deleteDoc(d.ref));

    // Nieuwe snapshot – uitgebreid payload
    await addDoc(SNAPSHOTS_COL, {
      label,
      ladderId: activeLadderId,
      ladderNaam: alleLadders.find(l => l.id === activeLadderId)?.naam,
      timestamp: Date.now(),
      datum: new Date().toLocaleString('nl-NL'),
      versie: '11.x', // schemaversie
      spelers: JSON.parse(JSON.stringify(state.spelers)),
      cfg: ladderData?.cfg || null,
      actievePartijen: JSON.parse(
        JSON.stringify(state.actievePartijen || [])
      ),
      spelerIds: ladderData?.spelerIds || [],
      standen, // {uid: {rank,...}, ...}
      nextId: state.nextId || 1
    });
  } catch(e) { console.error('Snapshot mislukt:', e); }
}
```

Verificatie stap 1: Speel een test-partij, kijk in Firebase Console in de snapshots/ collectie naar het nieuwste document. Velden `cfg`, `actievePartijen`, `spelerIds`, `standen` en `versie` moeten aanwezig zijn.

Stap 2: Herstel uitbreiden (herstelSnapshot)

File: js/beheer.js · regel ~443

De functie heeft sinds v11.31 al de standen-sync (optie A). Voeg nu de andere velden toe, met backwards-compatibility: oude snapshots zonder nieuwe velden moeten *niet crashen* en niet overschrijven met undefined.

```
async function herstelSnapshot(snapId) {
  try {
    const snapDoc = await getDoc(doc(db, 'snapshots', snapId));
    if (!snapDoc.exists()) { toast('Snapshot niet gevonden'); return; }
    const data = snapDoc.data();
    const ladderId = data.ladderId || activeLadderId;
    const ladderNaam = data.ladderNaam || ladderId;

    if (!confirm(...)) return;

    // Veiligheidsnet: backup huidige staat
    await slaSnapshotOp('■■ Voor herstel op ' +
      new Date().toLocaleString('nl-NL'));

    const { exists, data: ladderData } = await getLadderData(ladderId);
    if (!exists) { toast('Ladder niet gevonden'); return; }

    // Bouw payload – nieuwe velden alleen meenemen als snapshot ze heeft
    ladderData.spelers = data.spelers.map(s => ({
      ...s,
      partijen: s.partijen ?? 0,
      gewonnen: s.gewonnen ?? 0,
      prevRank: null
    }));
    if (data.cfg) ladderData.cfg = data.cfg;
    if (data.actievePartijen) ladderData.actievePartijen = data.actievePartijen;
    if (data.spelerIds) ladderData.spelerIds = data.spelerIds;
    if (data.nextId) ladderData.nextId = data.nextId;

    // Schrijf ladder-doc
    await setDoc(doc(db, 'ladders', ladderId), ladderData);

    // Update lokale state + cache (bestaande code blijft)
    if (ladderId === activeLadderId) {
      state.spelers = ladderData.spelers;
      state.actievePartijen = ladderData.actievePartijen || [];
      state.nextId = ladderData.nextId || state.nextId;
    }
    // ... alleLadders cache updaten ...

    // Standen-subcollectie – twee paden:
    if (data.standen) {
      // Snapshot HEEFT standen-data → schrijf direct terug
      const writes = [];
      for (const [uid, stand] of Object.entries(data.standen)) {
        writes.push(
          setDoc(doc(db, 'ladders', ladderId, 'standen', uid), stand)
        );
      }
      await Promise.all(writes);
    } else {
      // Legacy snapshot → val terug op afleiden uit spelers[] (optie A logica)
      // [bestaande v11.31 code: uid-lookup via _usersCache]
```

```
}

renderLadder();
toast(`Ladderstand "${ladderNaam}" hersteld ✓`);
closeModal('modal-snapshots');
} catch(e) { toast('Herstel mislukt: ' + e.message); }
}
```

Verificatie stap 2: Maak op /test/ een snapshot. Wijzig daarna ranks én cfg-instellingen. Doe een herstel. Controleer drie tabs: admin (state.spelers), ladder (standen subcollectie), beheer-instellingen (cfg). Alle drie moeten de oude waardes tonen.

Stap 3: Backwards compatibility testen

Bestaande snapshots in productie zijn van vóór v11.32 en hebben geen cfg, actievePartijen, standen, etc. Deze mogen niet crashen bij herstel.

- Test scenario: pak een legacy snapshot (zonder versie-veld) uit Firebase Console, doe herstel via UI
- Verificatie: geen crash, alleen spelers[] wordt teruggezet, andere staat blijft
- Verifieer dat de optie A standen-sync nog steeds draait als fallback (de uid-lookup via _usersCache)
- Test ook met snapshot die wel versie heeft maar standen leeg is — moet ook werken

Stap 4: UI weer zichtbaar maken

File: index.html · regel ~1430

De 'Ladder herstellen' kaart heeft sinds v11.30 `style="display:none"`. Verwijder dat attribuut. Behoud de `toggleAdminKaart-collapse` functie — kaart begint ingeklapt, gebruiker klikt om te openen.



Stap 5: Optionele uitbreidingen

Deze zijn niet strikt nodig voor optie B maar waren in eerdere analyse genoemd als gewenst. Kies bewust per item:

Uitbreiding	Werk	Aanbeveling
Handmatige snapshot-knop in beheer-tile	Klein — knop + <code>slaSnapshotOp('Handmatig - {datum}')daanroep</code>	Wel doen
Schema-versie nummer in payload (versie 11)	Tiël — al voorgesteld in stap 1	Wel doen
Ladder-filter strikter (snapshots zonder ladder-id pas filteren)	Klein — pas filteren in <code>laadSnapshots</code>	Wel doen
Transactie/rollback bij gedeeltelijke fout	Groot — Firestore batch writes, complexere rollback logica	Niet doen
30-dagen retentie aanpassen of configureren	Klein — aanmaken number 30 vervangen door const	Naar smaak

Stap 6: Versie- en cache-bump

- `js/app.js` — VERSION naar volgende patch (bv. v3.0.0-11.32)
- `sw.js` — `CACHE_VERSION` ophogen (bv. v120)
- Cache-bump is essentieel anders pakt SW de oude code op

Stap 7: Test op /test/ branch vóór productie

1. Bouw zip uit, upload naar /test/ folder van GitHub Pages
2. SW unregister + hard refresh in browser
3. Verifieer versie-badge: v3.0.0-11.32 TEST

4. Test scenario A: speel een partij, snapshot wordt automatisch gemaakt → bekijk in Firebase Console of payload klopt
5. Test scenario B: maak snapshot → wijzig ranks via stand-aanpassen → herstel → vergelijk admin én ladder tab
6. Test scenario C: pas cfg aan → maak snapshot → wijzig cfg opnieuw → herstel → cfg moet terug naar snapshot-waarde
7. Test scenario D: legacy snapshot uit Firebase Console terugzetten — mag niet crashen
8. Test scenario E: snapshot met actievePartijen → herstel → die partijen moeten weer in 'Bezig' staan

Risico's en valkuilen

Snapshot-grootte

Een snapshot wordt fors groter. Bij 70 spelers met cfg, actievePartijen en 70 standen-docs: schatting ~30-50 KB per snapshot. Bij gemiddeld 10 partijen per dag × 30 dagen = 300 snapshots × 40 KB = ~12 MB in totaal. Firestore kan dat aan binnen het gratis tier (1 GB), maar reads/writes kosten meer.

Mitigatie: houd 30-dagen retentie aan. Overweeg ook 'na X partijen' als extra opruimcriterium om snapshots tijdens druk seizoen niet te laten exploderen.

Geen transactie

Optie B doet meerdere Firestore writes (ladder-doc + N standen-docs). Als een deel slaagt en een deel faalt, zit data in een tussen-staat. Geen automatische rollback.

Mitigatie: de 'voor herstel'-snapshot die nu al wordt gemaakt biedt manuele recovery. Echte transactie zou Firestore batch writes vereisen — past niet binnen optie B scope.

Half-ingevulde partijen in actievePartijen

Als snapshot wordt gemaakt terwijl een partij half-ingevuld is, zit die half-ingevulde partij in de snapshot. Bij herstel komt die staat terug — scores van na het snapshot-moment gaan verloren bij die partij.

Mitigatie: niet kritiek omdat snapshots primair na bevestigde partijen worden gemaakt (niet tijdens). Wel waarschuwen in bevestiging-dialoog: *"Lopende partijen worden teruggezet naar staat van toen."*

Naam-collision in uid-lookup

De fallback voor legacy snapshots (zonder standen-veld) gebruikt naam→uid lookup. Als twee spelers dezelfde naam hebben, wordt die overgeslagen (zoals in optie A). Bij optie B is dit alleen relevant voor legacy snapshots; nieuwe snapshots hebben standen-data direct als {uid: ...} en hebben geen lookup nodig.

Lessons learned uit eerdere sessies

Bij implementatie van optie B, hou rekening met deze valkuilen die in eerdere fixes naar voren kwamen:

Twee datalagen sync. Elke wijziging die ranks/partijen/gewonnen raakt MOET beide lagen syncen. Dit is structureel (zie v11.27 → v11.28 → v11.31 historie). Optie B moet dit consistent blijven doen.

Cache-stale-issue. alleLadders[idx].data wordt async bijgewerkt door de live listener. Vertrouw deze cache niet direct na een schrijfactie. Lees zo nodig opnieuw uit Firestore.

Type-mismatch ids. s.id kan string of number zijn afhankelijk van legacy/uid status. Gebruik altijd String(s.id) in vergelijkingen, of expliciete typeof checks.

Service worker cache. Bij elke release CACHE_VERSION ophogen. Zonder dat blijft de oude code draaien in de browser, ook als versie-badge nieuwe waarde toont.

Firestore-rules. Bij schrijven naar standen/{uid} moet de auth-context kloppen. Beheerder-write is toegestaan in de huidige rules; verifieer dat dit zo blijft.

Stilzwijgende fouten. Catches met alleen `console.warn` maken bugs onzichtbaar. Voeg hoorbare feedback (toast) toe voor de gebruiker als kritieke writes falen.

Checklist voor uitvoering

Print deze pagina af of vink digitaal af tijdens de implementatie.

- Verse v11.31 zip lokaal uitgepakt als startbasis
- slaSnapshotOp uitgebreid met cfg, actievePartijen, spelerIds, standen, nextId, versie
- Verificatie: nieuwe snapshot in Firebase Console heeft alle velden
- herstelSnapshot uitgebreid: schrijft cfg/actievePartijen/spelerIds terug indien aanwezig
- herstelSnapshot: schrijft standen/{uid} direct als data.standen aanwezig
- herstelSnapshot: fallback naar uid-lookup als data.standen ontbreekt (legacy support)
- herstelSnapshot: bevestigingsdialoog meldt dat lopende partijen worden teruggezet
- Test op /test/: nieuwe snapshot terugzetten — alle drie tabs consistent
- Test op /test/: legacy snapshot terugzetten — geen crash, alleen spelers[] terug
- Test op /test/: cfg-wijziging na snapshot, daarna herstel — cfg terug
- Optioneel: handmatige snapshot-knop toegevoegd
- Optioneel: laadSnapshots filter aangescherpt (geen legacy-lekkage tussen ladders)
- UI 'Ladder herstellen' kaart weer zichtbaar (display:none weg in index.html)
- VERSION bumped (app.js)
- CACHE_VERSION bumped (sw.js)
- Productie-deploy via overschrijven root van GitHub Pages
- SW unregister + hard refresh getest na deploy
- Eerste echte gebruik geverifieerd: snapshot maken + bekijken in admin-tab werkt

Tot slot

Optie B is uitvoerbaar in één geconcentreerde sessie van 2-3 uur.

Optie A (al gedaan in v11.31) lost het meest acute probleem op zonder UI.

Een derde optie — volledig herstel inclusief uitslagen/-collectie en knockout-bracket

— is bewust buiten scope gehouden vanwege complexiteit.